Assignment 2

Monte Carlo Methods for Financial Applications

5MA178

Markus Ådahl

Sam Moeini and André Johansson
samo0171, anyp0002

Due Date: May 26, 2024

Contents

1 Task 1: Call and put option

1.1 Method for Task 1

1. Generate low-discrepancy sequences using the Sobol method.
2. Transform these sequences to normal distributions using the inverse transform method.
3. Simulate stock price paths at maturity for each strike price using the transformed sequences.
4. Calculate the call and put option payoffs at maturity.
5. Adjust the drift of the stock price process to target the relevant strike price more effectively.
6. Generate standard normally distributed random numbers.
7. Apply the adjusted drift to simulate stock price paths at maturity.
8. Calculate the call or put option payoffs using these paths.
9. Weight the payoffs by the likelihood ratio between the target and original distributions.
10. Average and discount these weighted payoffs to estimate the option prices.
11. Simulate option prices using different methods (Quasi-MC, Importance Sampling, SMC and CV).

1.2 Results: Task 1

The following table presents the calculated prices for call options using quasi-MC and importance sampling for various strike prices:

Table 1: Comparison of Call Option Pricing Methods

Method	K=70	K=100	K=130
Quasi-MC	32.230	9.410	1.355
Importance Sampling	-	-	1.357

Table 2: Comparison of Put Option Pricing Methods

Method	K=70	K=100	K=130
Quasi-MC	0.169	6.462	27.521
Importance Sampling	0.164	-	-

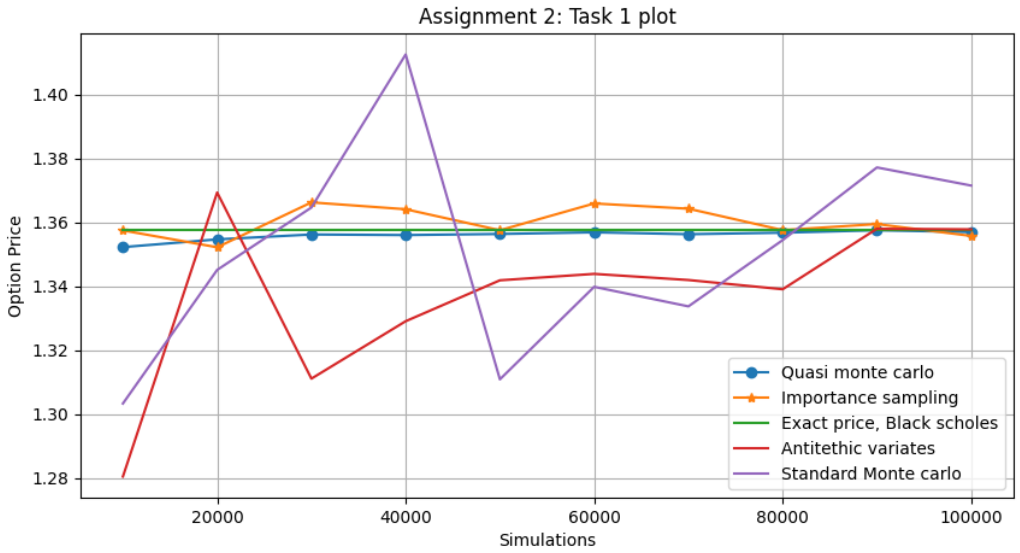

Figure 1: Comparison plot between how a call option with strike 130 is priced using methods from assignment 1, quasi-MC and Importance Sampling.

1.3 Discussion: Task 1

Based on the plot in c) which pricing method seems to be the most accurate?

Based on our plot showing different option pricing methods we could see that quasi monte carlo gave the most accurate pricing. The method gave a accurate result in fewer simulations and stayed most consistent with the price relative to number of simulations.

2 Task 2: Asian Call

2.1 Method for Task 2

1. Part 1: Define variables
2. Generate Sobol sequences to create low-discrepancy quasi-random numbers for each time step.
3. Convert these quasi-random numbers to normally distributed increments using the inverse normal function.
4. Simulate stock price paths using the geometric Brownian motion formula over m time steps.
5. Compute the arithmetic average of the simulated stock prices at each time step for each path.
6. Calculate the payoff for the Asian call option as the maximum of the average minus the strike price or zero.
7. Discount the average payoff and take the average across all paths.
8. Set up the initial conditions similar to the quasi-Monte Carlo method.
9. Adjust the drift of the stock price process.
10. Generate standard normally distributed random numbers for each time step.
11. Simulate stock price paths using the modified drift parameter to shift the mean towards the strike.
12. Compute the arithmetic average of stock prices at each time step for each path.
13. Calculate the Asian call option payoff as the maximum of the average minus the strike price or zero.
14. Apply the likelihood ratio correction to each payoff.
15. Discount and average the corrected payoffs to estimate the option price.
16. Simulate the option prices for the Asian call using the quasi-Monte Carlo, importance sampling methods and compare with SMC and CV.

2.2 Results: Task 2

Table 3: The following table presents the calculated prices for options using different methods

Option - Method	**K = 70**		**K = 100**		**K = 130**	
	m=12	m=52	m=12	m=52	m=12	m=52
Quasi-MC	30.701	30.597	5.640	5.353	0.128	0.09
Importance Sampling	-	-	-	-	0.132	0.094

Figure 2: Comparison plot between methods from assignment 1, quasi-MC and Importance Sampling for K = 130 and m = 52

3 Task 3: Basket call and put option on two underlying stocks

3.1 Method/ pseudocode for Task 3

1. Define variables.
2. Generate Sobol sequences to create low-discrepancy quasi-random numbers for each time step.
3. Convert these quasi-random numbers to normally distributed increments using the inverse normal function.
4. Simulate stock price paths using the geometric Brownian motion formula over the given number of time steps.

5. Compute the arithmetic average of the simulated stock prices at each time step for each path.
6. Calculate the payoff for the Asian call option as the maximum of the average minus the strike price or zero.
7. Discount the average payoff and take the average across all paths.
8. Set up the initial conditions similar to the quasi-Monte Carlo method.
9. Adjust the drift of the stock price process.
10. Generate standard normally distributed random numbers for each time step.
11. Simulate stock price paths using the modified drift parameter to shift the mean towards the strike.
12. Compute the arithmetic average of stock prices at each time step for each path.
13. Calculate the Asian call option payoff as the maximum of the average minus the strike price or zero.
14. Apply the likelihood ratio correction to each payoff.
15. Discount and average the corrected payoffs to estimate the option price.
16. Simulate the option prices for the Asian call using the quasi-Monte Carlo, importance sampling methods and compare with standard Monte Carlo and control variates.

3.2 Results: Task 3

Table 4: Basket option call prices

Method/K	K = 70	K = 100	K = 130
$\rho = 0$	32.079	7.181	0.341
$\rho = 0.5$	32.132	8.383	0.810

Table 5: Basket option put prices

Method/K	K = 70	K = 100	K = 130
$\rho = 0$	0.012	4.227	26.500
$\rho = 0.5$	0.064	5.429	26.969

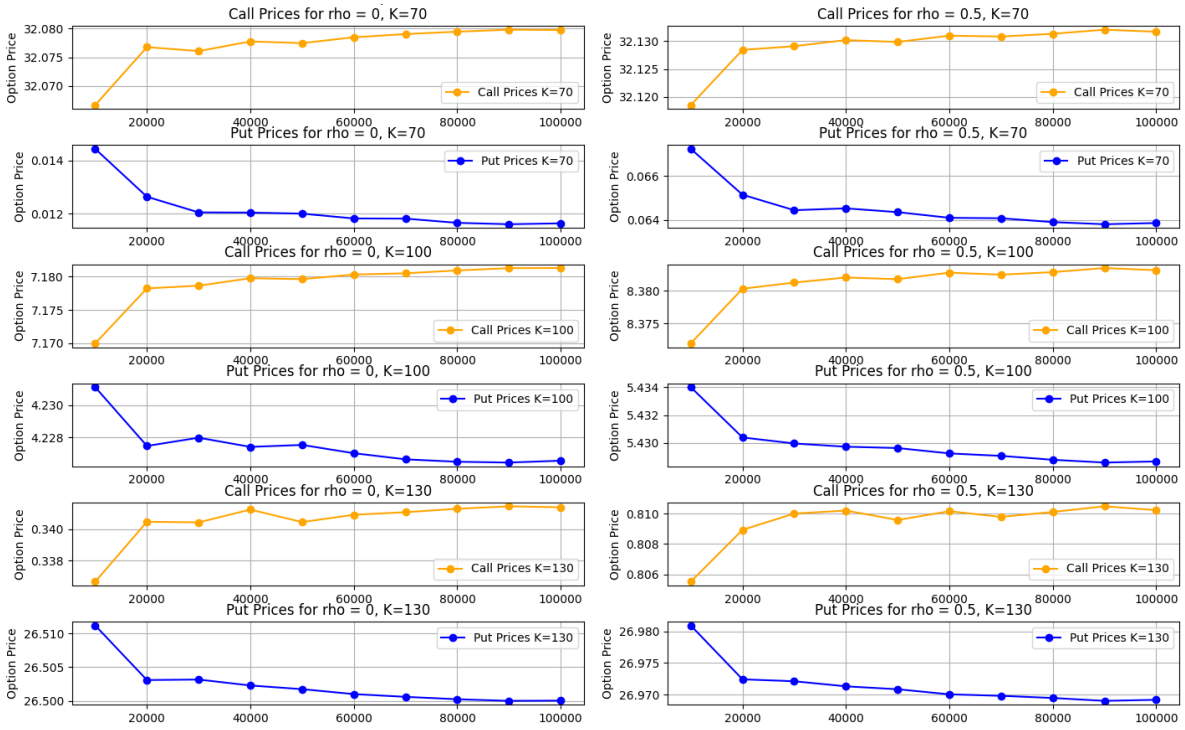

Figure 3: Plot showcasing the different movements for the prices during different correlations, strikeprices and number of simulations. (The plot wasnt specified but Markus told us to include it since it provided a good representation of the simulations.)

4 Task 4: Barrier option

4.1 Method/pseudocode for Task 4

1. Define variables
2. Generate two sets of normally distributed random variables. Use the Cholesky decomposition to introduce correlation between the two random variables, which will drive the stock price and volatility processes.
3. For each time step, update the variance using the Heston model dynamics, ensuring that the variance remains non-negative.
4. Update the stock price using the updated variance and the correlated random variables.
5. Check if the stock price hits the barrier.
6. Compute the option's payoff at maturity if the stock price has hit the barrier at any point during the life of the option.
7. Average the discounted payoffs across all simulated paths to estimate the price of the barrier option.
8. Repeat the simulation for different sets of volatility and correlation parameters.

4.2 Results: Task 4

The following tables presents the alculated prices assuming the stock follows the Heston model with different volatility's, rho's and step sizes.

Table 6: Results for $\rho = 0.7$

Periods	$\sigma = 20\%$	$\sigma = 40\%$
12	0.017	1.131
52	0.027	1.910

Table 7: Results for $\rho = -0.7$

Periods	$\sigma = 20\%$	$\sigma = 40\%$
12	0.056	1.284
52	0.102	2.015

4.3 Discussion: Task 4

Which correlation results in higher prices and why?

In the Heston model a negative correlation ($\rho = -0.7$) means that when the stock price goes down volatility tends to go up. Higher volatility increases the potential for large price movements making the option more valuable. Therefore we got higher prices for correlation $\rho = -0.7$.

5 Task 5: Call option on a zero-coupon bond

5.1 Method/ Pseudocode for Task 5

1. Define variables
2. Define functions for calculating the bond price.
3. Generate an array of maturities and calculate the zero-coupon bond prices and corresponding yields to maturity using the defined functions.
4. Print the calculated yields to maturity.
5. Define functions for calculating the forward rate
6. Calculate the price of a European call option using the defined functions.
7. Set up parameters for Monte Carlo simulation
8. Simulate interest rate paths using the Vasicek model for the given number of time periods and calculate the average interest rate at the end of the period.
9. Calculate the payoff of the European call option using the average interest rate obtained from the simulation.
10. Print the Monte Carlo price of the call option.

5.2 Results: Task 5

Year	1	2	3	4	5	6	7	8	9	10
Zcb-rate	0.0143	0.0148	0.0152	0.0155	0.0158	0.0161	0.0163	0.0165	0.0167	0.0168

Table 8: Zero-Coupon Bond Rates from Year 1 to Year 10

Table 9

Method	Price
Exact	0.106
Standard MC	0.107

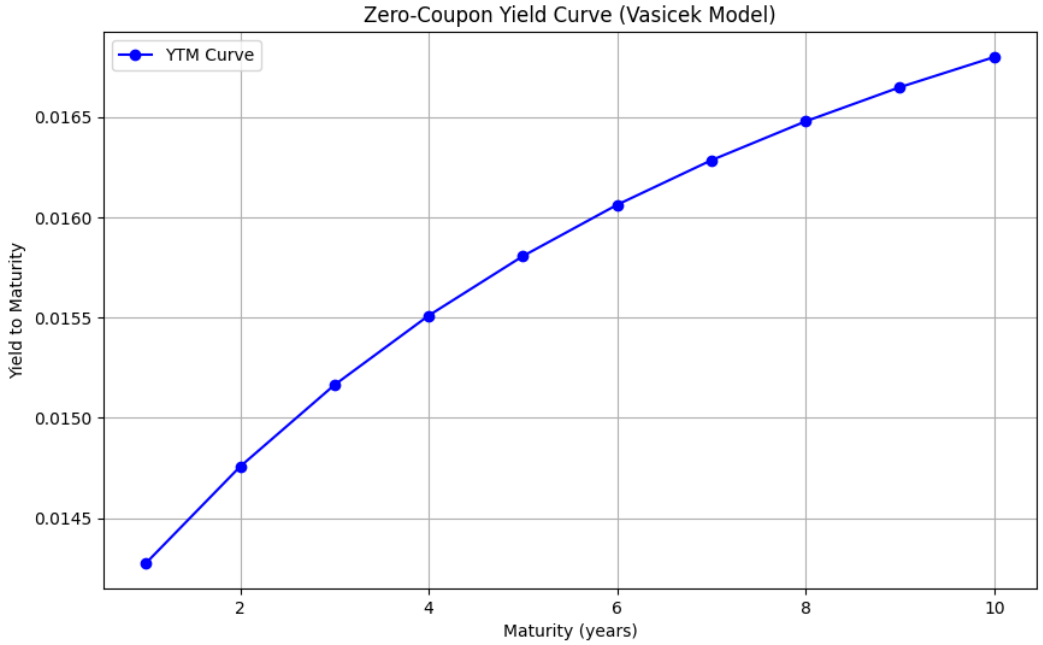

Figure 4: Plot showcasing the zero coupon intrest rate curve (YTM curve)

Python Code Sam Moeini and Andre Johansson May 26, 2024

6 Task 6: Adjustable rate mortgage with interest rate ceiling (ARMWIRC)

6.1 Method/ Pseudocode for Task 6

1. Define variables
2. Generate standard normal random numbers (`Z`) for each time step in each simulation path to represent interest rate increments.
3. Initialize an array (`rates`) to store the simulated interest rates for each path and time step, starting with the initial interest rate.
4. Simulate interest rate paths using the Vasicek interest rate model by iterating over time steps and updating the interest rates based on the Vasicek model equation. Ensure that the simulated rates are non-negative.
5. Cap the simulated interest rates at the specified ceiling value.
6. Compute the difference between the simulated rates and the capped rates to determine the portion above the ceiling.
7. Initialize arrays to store cash flows for the bank and the customer.
8. For each time step after the first one, calculate the discount factor using the cumulative sum of interest rates up to the current time step, and apply it to the

above-ceiling rates for the bank's cash flows and to the capped rates for the customer's cash flows.

9. Compute the total cash flows for the bank and the customer across all time steps for each simulation path.
10. Calculate the mean cash flows for the bank and the customer across all paths.
11. Compute the average interest rate over the last time step of each simulation path.
12. Compute the fair premium
13. Print the fair premium.

6.2 Results: Task 6

The simulated fair premium we got when we ran 100 000 simulated paths of the short rate was approximately 0.0021 which corresponds to 0.21 percent.

7 Python Code to solve all tasks

```
import numpy as np
import pandas as pd
from scipy.stats import norm, qmc
import matplotlib.pyplot as plt
from scipy.stats.mstats import gmean
from scipy.stats import norm, lognorm


#1a)

"""
S0 = 100
r  = 0.03
T = 1
sigma = 0.2
n_step = 1
mycall = 1.84
myput = (-2.26)
paths = 20000


def quasi1a(K):

    dt = T/n_step
    sampler = qmc.Sobol(1, scramble=False)

    points = sampler.random_base2(17)
```

```
    normal_points = norm.ppf(points)

    stock_pricesquasi = S0 * np.ones((paths, n_step))

    for j in range((paths)):
        stock_pricesquasi[j] = stock_pricesquasi[j] * np.exp(((r
            - 0.5 * sigma ** 2) * dt + ( sigma * np.sqrt(dt)) *
           normal_points[j]))

    payoffcallquasi = np.maximum(stock_pricesquasi - K,0) * np.
       exp(-r*T)
    payoffputquasi = np.maximum(K - stock_pricesquasi, 0) * np.
       exp(-r*T)

    pricecallquasi = np.mean(payoffcallquasi)
    priceputquasi = np.mean(payoffputquasi)
    return pricecallquasi, priceputquasi

print("call, put strike 70: ", quasi1a(70))
print("call, put strike 100: ", quasi1a(100))
print("call, put strike 130: ", quasi1a(130))

"""

#1b)

"""
S0 = 100
r  = 0.03
T = 1
sigma = 0.2
n_step = 1
mycall = 1.84
myput = (-2.26)
Kcall = 130
Kput = 70
paths = 20000

dt = T/n_step

z = np.random.standard_normal((paths, n_step))

stock_pricescallIS = S0 * np.ones((paths, n_step))
stock_pricesputIS = S0 * np.ones((paths, n_step))
payoffcallIS = np.ones((paths, n_step))
payoffputIS = np.ones((paths, n_step))
```

```
for i in range(paths):
    stock_pricescallIS[i] = stock_pricescallIS[i] * np.exp(((r -
        0.5 * sigma ** 2) * dt) + (mycall * sigma * np.sqrt(dt))
        + (sigma * np.sqrt(dt) * z[i]))
    stock_pricesputIS[i] = stock_pricesputIS[i] * np.exp(((r -
       0.5 * sigma ** 2) * dt) + (myput * sigma * np.sqrt(dt)) +
        (sigma * np.sqrt(dt) * z[i]))
    payoffcallIS[i] = np.maximum((stock_pricescallIS[i] - Kcall)
       ,0) * np.exp(-r*T) * np.exp((-mycall*(z[i]+mycall)) +
       (0.5*(mycall**2)))
    payoffputIS[i] = np.maximum((Kput - stock_pricesputIS[i]),
       0) * np.exp(-r*T) * np.exp((-myput*(z[i]+myput)) + (0.5*(
       myput**2)))

pricecallIS = np.mean(payoffcallIS)
priceputIS = np.mean(payoffputIS)

print("Call, strike 130: ", pricecallIS)
print("Put, strike 70: ", priceputIS)

"""

#1c)

"""
Vectorquasi = []
VectorIS = []
VectorAV = []
VectorSMC = []

S0 = 100
r  = 0.03
T = 1
sigma = 0.2
n_step = 1
mycall = 1.84
myput = (-2.26)
Kcall = 130
Kput = 70

for paths in range(10000, 100001, 10000):

    #Quasi

    dt = T/n_step
    sampler = qmc.Sobol(1, scramble=False)
    points = sampler.random_base2(17)
```

```
    normal_points = norm.ppf(points)

    stock_pricesquasi = S0 * np.ones((paths, n_step))
    for j in range((paths)):
        stock_pricesquasi[j] = stock_pricesquasi[j] * np.exp(((r
            - 0.5 * sigma ** 2) * dt + ( sigma * np.sqrt(dt)) *
           normal_points[j]))
    payoffcallquasi = np.maximum(stock_pricesquasi - Kcall,0) *
       np.exp(-r*T)

    pricecallquasi = np.mean(payoffcallquasi)


    #Importance sampling & SMC

    dt = T/n_step

    z = np.random.standard_normal((paths, n_step))

    stock_pricescallIS = S0 * np.ones((paths, n_step))
    stock_pricesputIS = S0 * np.ones((paths, n_step))
    StockpricesSMC = S0 * np.ones((paths, n_step))
    payoffcallIS = np.ones((paths, n_step))
    payoffputIS = np.ones((paths, n_step))
    payoffSMC = np.ones((paths, n_step))

    for i in range(paths):
        stock_pricescallIS[i] = stock_pricescallIS[i] * np.exp
           (((r - 0.5 * sigma ** 2) * dt) + (mycall * sigma * np
           .sqrt(dt)) + (sigma * np.sqrt(dt) * z[i]))
        StockpricesSMC[i] = StockpricesSMC[i] * np.exp(((r -
           (0.5 * (sigma ** 2))) * dt) + (sigma * np.sqrt(dt) *
           z[i]))
        payoffcallIS[i] = np.maximum((stock_pricescallIS[i] -
           Kcall),0) * np.exp(-r*T) * np.exp((-mycall*(z[i]+
           mycall)) + (0.5*(mycall**2)))
        payoffSMC[i] = np.maximum((StockpricesSMC[i] - Kcall ),
           0) * np.exp(-r*T)

    pricecallIS = np.mean(payoffcallIS)
    pricecallSMC = np.mean(payoffSMC)

    Vectorquasi.append(pricecallquasi)
    VectorIS.append(pricecallIS)
    VectorSMC.append(pricecallSMC)

# AV
```

```
for pathsAV in range(5000, 50001, 5000):

    z2 = np.random.standard_normal((pathsAV, n_step))
    z3 = -z2

    stock_pricesAVplus = S0 * np.ones((pathsAV, n_step))
    stock_pricesAVminus = S0 * np.ones((pathsAV, n_step))
    payoffsAVplus = np.ones((pathsAV, n_step))
    payoffsAVminus = np.ones((pathsAV, n_step))

    for h in range(pathsAV):
        stock_pricesAVplus[h] = stock_pricesAVplus[h] * np.exp
            (((r - (0.5 * (sigma * sigma))) * T) + (sigma * np.
            sqrt(T) * z2[h]))
        stock_pricesAVminus[h] = stock_pricesAVminus[h] * np.exp
            (((r - (0.5 * (sigma * sigma))) * T) + (sigma * np.
            sqrt(T) * z3[h]))
        payoffsAVplus[h] = np.maximum((stock_pricesAVplus[h] -
            Kcall), 0) * np.exp(-r * T)
        payoffsAVminus[h] = np.maximum((stock_pricesAVminus[h]-
            Kcall), 0) * np.exp(-r * T)

    payoffsAV = payoffsAVplus + payoffsAVminus

    resAV = np.sum(payoffsAV)/(2*(pathsAV))
    VectorAV.append(resAV)

# Black scholes
def BS(K):
        d1 = 1/(sigma * (np.sqrt(T))) * (np.log(S0/K) + (r + 0.5
            * (sigma * sigma))*(T))
        d2 = d1 - sigma * np.sqrt(T)
        call = S0 *  norm.cdf(d1) - np.exp((-r)* (T)) * K * norm
            .cdf(d2)
        return call

BSvector = np.ones(10) * BS(130)


# 1c) plot

plt.figure(figsize=(10, 5))
plt.plot(range(10000, 100001, 10000), Vectorquasi, label='Quasi
    monte carlo', marker = "o")
plt.plot(range(10000, 100001, 10000), VectorIS, label='
    Importance sampling', marker = "*")
plt.plot(range(10000, 100001, 10000), BSvector, label='Exact
    price, Black scholes')
```

```
plt.plot(range(10000, 100001, 10000), VectorAV, label='
    Antitethic variates')
plt.plot(range(10000, 100001, 10000), VectorSMC, label='Standard
    Monte carlo')
plt.title('Assignment 2: Task 1 plot')
plt.xlabel('Simulations')
plt.ylabel('Option Price')
plt.legend()
plt.grid(True)
plt.show()

"""



"""
# 2 a)

def q2(K,M):

    r = 0.03
    sigma = 0.2
    s0 = 100
    T2 = 1

    n = 20000

    dt = T2 / M
    sampler = qmc.Sobol(M, scramble=False)
    points = sampler.random_base2(15)
    normal_points = norm.ppf(points)

    stock_prices = s0 * np.ones((n, M + 1))
    stock_prices[:,0] = s0

    for i in range((n)):
        for t in range(1, M + 1):
            stock_prices[i, t] = stock_prices[i, t - 1] * np.exp
                (((r - 0.5 * sigma ** 2) * dt + (sigma * np.sqrt(
                dt)) * normal_points[i, t-1]))


    C2 = np.mean(stock_prices[:, 1:], axis = 1)
    payoffsSMC = np.maximum(C2-K,0) * np.exp(-r * T2)

    SMC = np.mean(payoffsSMC)

    return SMC
```

```
print("Quasi-MC for strike 70 and m = 12 ", q2(70, 12))
print("Quasi-MC for strike 100 and m = 12 ", q2(100, 12))
print("Quasi-MC for strike 130 and m = 12 ", q2(130, 12))
print("Quasi-MC for strike 70 and m = 52 ", q2(70, 52))
print("Quasi-MC for strike 100 and m = 52 ", q2(100, 52))
print("Quasi-MC for strike 130 and m = 52 ", q2(130, 52))

"""
"""

#2 b)

def q2b(m):
    S0 = 100
    r  = 0.03
    T = 1
    sigma = 0.2
    mycall = 5/m
    paths = 20000
    K = 130

    dt = T/m


    stock_pricescallIS = S0 * np.ones((paths, m+1 ))
    LR = np.ones((paths))

    for t in range(1, m + 1):
        z = np.random.standard_normal(paths)
        z2 = z + mycall
        stock_pricescallIS[: , t] = stock_pricescallIS[:, t-1] *
            np.exp(((r - 0.5 * sigma ** 2) * dt) + (sigma * np.
           sqrt(dt) * z2))
        LR = LR * np.exp(-mycall*z2 + 0.5*(mycall**2))


    C = np.mean(stock_pricescallIS[:, 1:], axis=1)
    payoffcallIS = np.maximum((C - K),0) * np.exp(-r*T) * LR
    pricecallIS = np.mean(payoffcallIS)

    return pricecallIS


print( "Asian call option price with strike 130 and m = 12 : ",
   q2b(12))
print( "Asian call option price with strike 130 and m = 52 : ",
   q2b(52))
```

```
"""


"""

# 2c)

#Importance sampling m = 52, K = 130

ISvector = []
QMCvector = []
SMCvector = []
CVvector = []


for paths in range (10000, 100001, 10000):

    S0 = 100
    r  = 0.03
    T = 1
    sigma = 0.2
    m = 52
    mycall = 5/m
    K = 130

    dt = T/m

    stock_pricescallIS = S0 * np.ones((paths, m+1 ))
    LR = np.ones((paths))

    for t in range(1, m + 1):
        z = np.random.standard_normal(paths)
        z2 = z + mycall
        stock_pricescallIS[: , t] = stock_pricescallIS[:, t-1] *
            np.exp(((r - 0.5 * sigma ** 2) * dt) + (sigma * np.
           sqrt(dt) * z2))
        LR = LR * np.exp(-mycall*z2 + 0.5*(mycall**2))

    C = np.mean(stock_pricescallIS[:, 1:], axis=1)
    payoffcallIS = np.maximum((C - K),0) * np.exp(-r*T) * LR
    pricecallIS = np.mean(payoffcallIS)
    ISvector.append(pricecallIS)




    #Qusdi-MC M = 52, K = 130
```

```
    r = 0.03
    sigma = 0.2
    s0 = 100
    T2 = 1
    M = 52
    dt = T2 / M

    sampler = qmc.Sobol(M, scramble=False)
    points = sampler.random_base2(17)
    normal_points = norm.ppf(points)

    stock_prices = s0 * np.ones((paths, M + 1))
    stock_prices[:,0] = s0

    for i in range((paths)):
        for t in range(1, M + 1):
            stock_prices[i, t] = stock_prices[i, t - 1] * np.exp
               (((r - 0.5 * sigma ** 2) * dt + (sigma * np.sqrt(
               dt)) * normal_points[i, t-1]))


    C2 = np.mean(stock_prices[:, 1:], axis = 1)
    payoffsSMC = np.maximum(C2-K,0) * np.exp(-r * T2)
    SMC = np.mean(payoffsSMC)
    QMCvector.append(SMC)




# SMC, Comtrolvariate for m = 52, K = 130


    m = 52
    K = 130
    r = 0.03
    sigma = 0.2
    s0 = 100
    T2 = 1

    ti = []

    for i in range(1,m+1,1):
        t = (1/m)* i
        ti.append(t)

        T = np.sum(ti)/m
```

```
    Maturity = ti[-1]

    V = []

for ii in range(1,m+1,1):
    sigmaVariable = (2*ii-1) * ti[-1 + 1 - ii]
    V.append(sigmaVariable)
    Vinsigmaformula = np.sum(V)


sigmahattsquare = (np.square(sigma)/(np.square(m)*T)) *
   Vinsigmaformula

d = (0.5 * np.square(sigma)) - (0.5 * sigmahattsquare)

d1 = (1/(np.sqrt(sigmahattsquare) * (np.sqrt(T)))) * (np.log
   (s0/K) + (r - d + (0.5 * sigmahattsquare))*T)
d2 = d1 - (np.sqrt(sigmahattsquare) * np.sqrt(T))
ST1 = np.exp((-r)* (Maturity-T)) * (np.exp((-d) * T) * s0 *
    norm.cdf(d1) -  np.exp((-r) * T) * K * norm.cdf(d2))

dt = T2 / m

z = np.random.standard_normal((paths, m + 1))

stock_prices = s0 * np.ones((paths, m + 1))
stock_prices[:,0] = s0

for o in range((paths)):
    for t in range(1, m + 1):
        stock_prices[o, t] = stock_prices[o, t - 1] * np.exp
           (((r - 0.5 * sigma ** 2) * dt + (sigma * np.sqrt(
           dt)) * z[o, t]))

C1 = gmean(stock_prices[:, 1:], axis = 1)
payoffsCV = np.maximum(C1-K, 0) * np.exp(-r * T2)

C2 = np.mean(stock_prices[:, 1:], axis = 1)
payoffsSMC = np.maximum(C2-K,0) * np.exp(-r * T2)

Geosim = np.mean(payoffsCV)
SMC = np.mean(payoffsSMC)

#b = (np.cov(dispayoffCV,dispayoffSMC)[0,1]) / np.var(
   dispayoffCV)
b = sum((payoffsCV - Geosim) * (payoffsSMC - SMC)) / sum((
   payoffsSMC - Geosim) ** 2)
```

```
        Controlvariate = (SMC - b*(Geosim - ST1))


        SMCvector.append(SMC)
        CVvector.append(Controlvariate)



# plot 2 c)
plt.figure(figsize=(10, 5))
plt.plot(range(10000, 100001, 10000), ISvector, label='
    Importance sampling')
plt.plot(range(10000, 100001, 10000), QMCvector, label='Quasi -
    Monte carlo')
plt.plot(range(10000, 100001, 10000), SMCvector, label='Standard
     Monte carlo')
plt.plot(range(10000, 100001, 10000), CVvector, label='Control
    variate')
plt.title('Assignment 2: Task 2 plot')
plt.xlabel('Simulations')
plt.ylabel('Option Price')
plt.legend()
plt.grid(True)
plt.show()


"""

"""

# 3

S0 = 100
r  = 0.03
T = 1
sigma = 0.2
n_step = 1
K = [70, 100, 130]


rho_corr_0 = np.eye(2)
rho_corr_05 = (np.ones(2) + np.eye(2)) / 2
rho_list = [rho_corr_0, rho_corr_05]
rho_labels = ['rho = 0', 'rho = 0.5']


fig, axes = plt.subplots(6, 2, figsize=(14, 12))
```

```
epsilon = 1e-200
for rho_index, rho in enumerate(rho_list):
    for k_index, k in enumerate(K):
        call_prices = []
        put_prices = []
        num_paths = []

        for paths in range(10000, 100001, 10000):
            dt = T / n_step


            sampler = qmc.Sobol(d=2, scramble=False)
            points = sampler.random_base2(m=int(np.log2(paths)) +
                1)[:paths]


            points = np.clip(points, epsilon, 1 - epsilon)

            normal_points = norm.ppf(points)


            L = np.linalg.cholesky(rho)
            normal_points_corr = np.dot(normal_points, L.T)

            stock1_prices = S0 * np.ones((paths, n_step))
            stock2_prices = S0 * np.ones((paths, n_step))


            for i in range(paths):
                stock1_prices[i] *= np.exp((r - 0.5 * sigma ** 2)
                    * dt + sigma * np.sqrt(dt) *
                   normal_points_corr[i, 0])
                stock2_prices[i] *= np.exp((r - 0.5 * sigma ** 2)
                    * dt + sigma * np.sqrt(dt) *
                   normal_points_corr[i, 1])


            average_stockprice = (stock1_prices + stock2_prices)
               / 2
            payoffcall = np.maximum(average_stockprice - k, 0) *
               np.exp(-r * T)
            payoffput = np.maximum(k - average_stockprice, 0) *
               np.exp(-r * T)


            pricecall = np.mean(payoffcall)
            priceput = np.mean(payoffput)
```

```
            call_prices.append(pricecall)
            put_prices.append(priceput)
            num_paths.append(paths)

        pricecalls = np.mean(pricecall)
        priceputs = np.mean(priceput)
        print(f"CALL option price is {pricecalls} for STRIKE {k}
           and correlation {rho_labels[rho_index]}")
        print(f"PUT option price is {priceputs} for STRIKE {k}
           and correlation {rho_labels[rho_index]}")


        ax_call = axes[k_index * 2, rho_index]
        ax_call.plot(num_paths, call_prices, label=f'Call Prices
           K={k}', color='orange', marker='o')
        ax_call.set_title(f'Call Prices for {rho_labels[rho_index
           ]}, K={k}')
        ax_call.set_ylabel('Option Price')
        ax_call.legend()
        ax_call.grid(True)

        ax_put = axes[k_index * 2 + 1, rho_index]
        ax_put.plot(num_paths, put_prices, label=f'Put Prices K={
           k}', color='blue', marker='o')
        ax_put.set_title(f'Put Prices for {rho_labels[rho_index
           ]}, K={k}')
        ax_put.set_ylabel('Option Price')
        ax_put.legend()
        ax_put.grid(True)




fig.tight_layout()
fig.suptitle('Option Prices (Call and Put) for Different
   Correlations and Strikes', y=1.02)
plt.show()

"""

"""

# 4a)


def q4a(m, sigma, rho):
```

```
    s0 = 100
    K = 100
    T = 1
    r = 0.03
    B = 80
    dt = 1/m
    a = 0.1
    b = sigma**2
    sigmav = 0.1
    n = 20000

    s = s0 * np.ones((n, m+1 ))
    v = b * np.ones((n,m+1))

    z1 = np.random.standard_normal((n, m+1))
    z2 = np.random.standard_normal((n, m+1))
    Covariance_matrix = np.array([[1, rho], [rho, 1]])
    V = np.linalg.cholesky(Covariance_matrix)
    x1 = 1*z1
    x2 = rho*z1 + 0.71414284*z2

    for t in range(1, m+1):
        v[:, t] = np.maximum(v[:, t-1] + a*(b - v[:, t-1])*dt
            + sigmav*np.sqrt(v[:, t-1]*dt)*x1[:,t-1], 0)
        s[:, t] = s[:, t-1] * (1 + r*dt + np.sqrt(v[:, t-1]*dt
            ) * x2[:,t-1])

    payoff = np.zeros(n)
    for i in range(n):
        if np.any(s[i] <= B):
            payoff[i] = np.exp(-r*T) * np.maximum(s[i, -1] - K,
                0)

    c = np.mean(payoff)

    return c


print("m=12, sigma = 0.2, rho = 0.7, ", q4a(12,0.2,0.7))
print("m=52, sigma = 0.2, rho = 0.7, ", q4a(52,0.2,0.7))
print("m=12, sigma = 0.4, rho = 0.7, ", q4a(12,0.4,0.7))
print("m=52, sigma = 0.4, rho = 0.7, ", q4a(52,0.4,0.7))

print("m=12, sigma = 0.2, rho = -0.7, ", q4a(12,0.2,-0.7))
print("m=52, sigma = 0.2, rho = -0.7, ", q4a(52,0.2,-0.7))
print("m=12, sigma = 0.4, rho = -0.7, ", q4a(12,0.4,-0.7))
print("m=52, sigma = 0.4, rho = -0.7, ", q4a(52,0.4,-0.7))
```

```
"""

"""

# 5a

K = 0.8; T1 = 5; T2 = 10; r0 = 0.0137; a = 0.2; b = 0.02; sigma
    = 0.01



def B(t, T, a):
    return (1 - np.exp(-a * (T - t))) / a


def A(t, T, a, b, sigma):
    B_val = B(t, T, a)
    term1 = ((B_val - (T-t)) * (a**2 * b - 1/2 * sigma**2)) / (a
        **2)
    term2 = (sigma ** 2) * (B_val ** 2) / (4 * a)
    return term1 - term2


def zero_coupon_bond_price(t, T, r, a, b, sigma):
    B_val = B(t, T, a)
    A_val = A(t, T, a, b, sigma)
    return np.exp(A_val - B_val*r)


def yield_to_maturity(t, T, bond_price):
    return -np.log(bond_price) / (T - t)


maturities = np.linspace(1, 10, 10)



bond_prices = [zero_coupon_bond_price(0, T, r0, a, b, sigma) for
     T in maturities]
ytm = [yield_to_maturity(0, T, price) for T, price in zip(
    maturities, bond_prices)]
print("Task a: ",ytm)
```

```
# Plot
plt.figure(figsize=(10, 6))
plt.plot(maturities, ytm, label='YTM Curve', color='blue',
    marker = 'o')
plt.xlabel('Maturity (years)')
plt.ylabel('Yield to Maturity')
plt.title('Zero-Coupon Yield Curve (Vasicek Model)')
plt.grid(True)
plt.legend()
plt.show()



# 5b
def sigma_p(T1,T2,t,a,sigma):
    return 1/a * (1-np.exp(-a*(T2-T1))) * np.sqrt((sigma**2)/(2*a
        ) * ((1-np.exp(-2*a*(T1-t)))))


def d(T1,T2,t,a,sigma,r):
    sigma_p_val = sigma_p(T1,T2,t,a,sigma)
    p_t_T1 = zero_coupon_bond_price(t,T1,r,a,b,sigma)
    p_t_T2 = zero_coupon_bond_price(t,T2,r,a,b,sigma)
    return 1/sigma_p_val * np.log(p_t_T2 / (p_t_T1 * K)) + 1/2 *
        sigma_p_val
def c(t,T1,K,T2,a,r):
    d_val = d(T1,T2,t,a,sigma,r)
    sigma_p_val = sigma_p(T1,T2,t,a,sigma)
    p_t_T1 = zero_coupon_bond_price(t,T1,r,a,b,sigma)
    p_t_T2 = zero_coupon_bond_price(t,T2,r,a,b,sigma)
    return p_t_T2 * norm.cdf(d_val) - p_t_T1 * K * norm.cdf(d_val
        - sigma_p_val)




price = c(0,T1,K,T2,a,r0)
print(f"Task b: {price:.4f}")



# 5c


periods = 52
dt = 1 / periods
```

```
paths = 20000
Z = np.random.standard_normal((paths, periods*T1))
rates = np.zeros((paths, periods*T1))
rates[:,0] = r0


for t in range(1, periods * T1 - 1):
    rates[:, t+1] = rates[:, t] + a * (b - rates[:, t]) * dt +
        sigma * np.sqrt(dt) * Z[:, t+1]

average_r = np.mean(rates[:,-1])
payoff2 = c(0,T1,K,T2,a,average_r)



print(f"Task c: Monte Carlo price of the call option: {payoff2
    :.4f}")

"""

"""

# 6
paths = 100000
a = 0.2
b = 0.02
r0 = 0.0137 # Starting intrest rate
sigma = 0.01 # Volatility
ceiling = 0.023 # 2.3% intrest rate ceiling
T = 5 # Five years
periods = 12 # Monthly rate
dt = 1/periods # Time steps


Z = np.random.standard_normal((paths, periods*T))
rates = np.zeros((paths, periods*T))
rates[:,0] = r0
for t in range(0, periods * T-1):
    rates[:, t+1] = rates[:, t] + a * (b - rates[:, t]) * dt +
        sigma * np.sqrt(dt) * Z[:, t+1]
    rates[:, t + 1] = np.maximum(rates[:, t + 1], 0)


capped_rates = np.minimum(rates, ceiling)
above_rates = rates - capped_rates


bank_cashflow = np.zeros((paths, periods*T))
```

```
customer_cashflow = np.zeros((paths, periods*T))


for t in range(periods*T):
    discount_factor = np.exp(-rates[:, :t].sum(axis=1) * dt)
    discount_factor = np.exp(-r0*t*dt)
    bank_cashflow[:,t] = above_rates[:,t]  * discount_factor
    customer_cashflow[:,t] = capped_rates[:,t] * discount_factor


total_bank_cashflows = np.zeros((paths))
total_customer_cashflows = np.zeros((paths))
for p in range(paths):
    total_bank_cashflows[p] = np.sum(bank_cashflow[p,:])
    total_customer_cashflows[p] = np.sum(customer_cashflow[p,:])




bank_mean_cashflow = np.mean(total_bank_cashflows)
customer_mean_cashflow = np.mean(total_customer_cashflows)
average_r = np.mean(rates[:,-1])
fair_premium = (bank_mean_cashflow / customer_mean_cashflow)/(
    periods*T) * np.exp(-average_r * 5)
print("Fair Premium", fair_premium)

"""
```